

# SDS Exercise Sheet 3: Natural Language Understanding

Nurul Lubis

Release 5.5.2026 — Due 1.6.2026

## 1 Introduction

In this exercise you are going to explore the NLU module of a dialogue system. The objective of this practical are as follows:

- Able to formalize the task of natural language understanding into a machine learning problem
- Able to train a model to perform language understanding with a reasonable performance

## 2 Preliminaries: Optimizing neural networks with supervised learning

### 2.1 Data

In **supervised machine learning**, we typically partition the dataset into three subsets: training, development (dev/validation), and test. Each of these partitions serves a different purpose:

- The training set is used to fit the model parameters.
- The dev set is used to tune hyperparameters—settings that control the learning process, such as learning rate, model architecture, or regularization strength.
- The test set provides an unbiased evaluation of the model's performance on unseen data.

This separation helps detect and prevent **overfitting**, which occurs when a model learns patterns specific to the training data, including noise or outliers, and fails to generalize well to new inputs.

### 2.2 Training

Training a neural network involves two main steps: the **forward pass** and **backpropagation**.

**Forward pass** In the forward pass, input data is propagated through the layers of the network to produce output predictions. The predictions are compared to the true labels using a **loss function**. For classification tasks, a common choice is the **cross-entropy loss**:

$$\mathcal{L}_{\text{cross-entropy}} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where:

- $C$  is the number of classes,
- $y_i$  is the ground truth label (often one-hot encoded),
- $\hat{y}_i$  is the predicted probability for class  $i$ .

**Backpropagation and gradient descent** In the backpropagation step, gradients of the loss with respect to the model parameters are computed using the chain rule. These gradients are used to update the parameters via **gradient descent**:

$$\theta \leftarrow \theta - \alpha \cdot \nabla_{\theta} \mathcal{L}$$

where:

- $\theta$  represents the model parameters (e.g., weights and biases),
- $\alpha$  is the learning rate,
- $\nabla_{\theta} \mathcal{L}$  is the gradient of the loss with respect to the parameters.

This process is repeated iteratively over multiple training steps (epochs) until the model converges or performance on the dev set stops improving.

## 2.3 Loss function vs. evaluation metrics

We optimize a differentiable loss function (like cross-entropy) during training because it enables gradient-based updates via backpropagation. In contrast, evaluation metrics (e.g. F1 score) are often non-differentiable and cannot be directly optimized using gradient descent. Instead, we use the loss as a surrogate that correlates with these metrics and allows for efficient optimization.

## 2.4 Training Configuration Files

Training pipelines often use external **configuration files** (e.g., YAML or JSON) to manage experiment settings. These files make it easy to adjust and track **hyperparameters** such as learning rate, model architecture, context length, without modifying the training code directly. This improves reproducibility and facilitates experimentation.

One commonly specified hyperparameter is the choice of a **pretrained model** as a starting point for training. Instead of training all weights from scratch, we can initialize a model with parameters learned from a large dataset (e.g., BERT). This practice, known as **fine-tuning**, allows the model to leverage general features and adapt them to the new task with less data and training time.

## 3 Exercise

### 3.1 Codebase

Use the same repository used in exercise 2.

### 3.2 Tasks

The script used in tasks 1 and 2 is

`convlab3/convlab/nlu/jointBERT/train.py`

**Task 1: Data loading for NLU training** You will implement the data loading for NLU training, ensuring that intent and slot labels are correctly mapped and processed. First, execute the data preprocessing step following the file: `convlab3/convlab/nlu/jointBERT/unified_datasets/README.md`.

After that, translate the following step by step instructions into code, inside the box labeled "TODO NLU\_TASK1".

- Load the intent and slot vocabularies
  - Read `intent_vocab.json` and `tag_vocab.json` from `data_dir`.
  - Store them in `intent_vocab` and `tag_vocab`.
- Initialize the `Dataloader` object
- Load training, validation, and test data into the dataloader
  - Iterate over the following data splits `{'train', 'val', 'test'}`.
  - Read `<split>.data.json` files.
  - Call `dataloader.load_data()` with the correct parameters from `config`
  - At the end, `dataloader.data` should be a dictionary containing train, validation, and test splits

**Task 2: Forward pass and backpropagation** You will implement the forward pass and loss calculation of the NLU module. Consult the NLU model class to figure out the input and output of the forward pass. Translate the following step by step instructions into code, inside the box labeled "TODO NLU\_TASK2".

- Use data from `batched_data` as input to the `model_forward` function
- Use the output of the forward pass function to compute `loss`. The final loss is the sum of slot tagging and intent prediction loss.

**Task 3: Evaluation metrics** You will implement the computation of precision, recall, and F1 to evaluate the NLU model. Translate the following instructions into code, inside the box labeled "TODO NLU\_TASK3". The script used in this task is

`ConvLab3/convlab/nlu/evaluate_unified_datasets.py`.

- Given a dictionary `metrics`, where each key corresponds to a slot type, update the dictionary by computing precision, recall, and F1-score for each metric. For undefined cases, assign 0 to the metric.
- The following dictionary is expected in the end:
 

```
metrics = {
  metric_1 = {'precision': 0, 'recall': 0, 'f1': 0},
  metric_2 = {'precision': 0, 'recall': 0, 'f1': 0},
  ..., accuracy : 0
}
```

**Task 4: Train an NLU** Lastly, train the NLU following the `README.md` of the `jointBERT` module. The config file is provided under

```
convlab3/convlab/nlu/jointBERT/unified_datasets/
configs/simplemultiwoz21_user_context3.json
```

The default config uses `prajjwal1/bert-mini` as the pretrained starting point, which is smaller than the commonly used `bert-base-uncased` and should thus be able to run on CPUs. You are free to modify any hyperparameters in the config file to improve your model performance. If you do, make sure to push the final config to your git repository.

### 3.3 Results and Discussion

In your final report, write a short report covering, but not limited to, the following points. Additional insights, analysis, and discussion are highly encouraged.

- Explain the model input and output used to train the BERTNLU model. In the dataset used, how many samples are there in the training, dev, and test sets?
- What configurations and hyperparameters can be set in training the model? Describe the setting you have chosen and its reasoning.
- Present the precision, recall, and F1 score of your model on the dev and test sets in a table. If you tried more than one configuration, compare them.
- What are the limitations of this model? Discuss improvements that can be done.

Push your completed scripts (and optionally config file) into your git repository and put the link to your repository in your report.